Лабораторная работа 2 — часть «Деревья и ансамбли в sklearn по CRISP-DM»

Аналитические и вычислительные задания

Содержание

Назначение работы 1

Данные и этика применения 1

Как выполнять работу 1

Маршрут 1

Что сдавать 2

Краткая справка sklearn 2

Назначение работы

В части, посвящённой ансамблям, вы проходите полный, но компактный цикл CRISP-DM на реальной задаче бинарной классификации. Нужно оценить вероятность того, что клиент банка оформит срочный вклад после маркетинговой кампании. Работа учит не только вызывать модели, но и отделять признаки, доступные **до** контакта с клиентом, от сведений, которые становятся известны после него.

Работа рассчитана на две пары и самостоятельное завершение. Результат — два сравнения: честное сравнение исходных моделей на кросс-валидации и единственная финальная оценка выбранных моделей на закрытой тестовой выборке.

Основная теория приведена в лонгриде «Деревья решений и ансамбли». По API обращайтесь к руководству по деревьям scikit-learn, ансамблям, типичным ошибкам и поиску гиперпараметров.

Данные и этика применения

Используется `data/bank-full.csv` — полный вариант набора UCI Bank Marketing: 45 211 строк и 16 входных признаков. Целевая переменная `y` показывает, согласился ли клиент на срочный вклад. Источник: UCI Machine Learning Repository, лицензия CC BY 4.0; подробности и контрольные суммы перечислены в `data/DATASETS.md`.

В исходном наборе `duration` — длительность уже состоявшегося звонка. Её нельзя использовать в задаче, где решение о том, кому звонить, принимают **до** звонка: значение недоступно в момент прогнозирования и создаёт утечку цели. В основной модели этот столбец исключается. В конце работы есть отдельный контролируемый опыт, который демонстрирует проблему только на обучающей части данных.

Набор описывает исторические решения одной организации, а не универсальное правило обращения с людьми. Модель не должна автоматически принимать финансовое решение, заменять человека или использоваться без проверки справедливости, стоимости ошибок и актуальности данных.

Как выполнять работу

Выполняйте студенческий ноутбук сверху вниз. Нумерованные разделы ноутбука соответствуют фазам CRISP-DM.

- Markdown-ячейки объясняют цель шага, ожидаемый тип результата и источник для самостоятельного чтения.
- В кодовых ячейках помечены места, которые нужно дописать. В тексте заданий нет скрытых ответов.
- Не подбирайте параметры по тестовой выборке. Кросс-валидация проводится только на обучающей части.
- Для всех случайных операций используйте `random_state=42`.

Маршрут

1. Сформулировать решение, момент прогнозирования и метрику успеха.
2. Проверить структуру, типы, баланс классов и смысл признаков.
3. Подготовить признаки в `Pipeline`: преобразовать цель, исключить `duration`, обработать код `pdays=-1`, закодировать категории.
4. Построить baseline: prior baseline, CART, bagging, Random Forest и Gradient Boosting.
5. Настроить все tree-модели: дерево — через компактный `GridSearchCV`, ансамбли — через `RandomizedSearchCV` с тремя фолдами.
6. Один раз оценить отобранные модели на test, сравнить PR-AUC, ROC-AUC, precision, recall, F1 и balanced accuracy.
7. Интерпретировать важности признаков и проверить их permutation-оценкой.
8. Показать, почему `duration` создаёт утечку, и оформить краткую модельную карточку.

Что сдавать

Сдаётся заполненный студентческий ноутбук с сохранёнными результатами выполнения ячеек. Перед отправкой проверьте:

- test не использовался для выбора модели, параметров или признаков;
- `duration` отсутствует в основной матрице признаков;
- код `pdays=-1` не истолкован как «минус один день»;
- preprocessing находится внутри `Pipeline` и проходит кросс-валидацию вместе с моделью;
- таблица baseline содержит все пять моделей;
- для CART, bagging, Random Forest и Gradient Boosting выполнен подбор параметров;
- окончательная таблица содержит все требуемые метрики и поясняет выбор основной;
- важность по impurity не выдана за причинный эффект и сопоставлена с permutation importance;
- все ячейки выполняются сверху вниз без ошибок после заполнения кода.

Краткая справка sklearn

```
Pipeline([("preprocessor", preprocessor), ("model", estimator)])
ColumnTransformer([...])
train_test_split(X, y, stratify=y, random_state=42)
StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
GridSearchCV(pipeline, grid, scoring="average_precision", cv=cv)
RandomizedSearchCV(pipeline, distributions, n_iter=6, scoring="average_precision", cv=cv)
permutation_importance(pipeline, X_test, y_test, scoring="average_precision")
```

`average_precision` выбран основной метрикой отбора, потому что положительный класс редок. Он измеряет качество ранжирования положительных объектов при разных порогах; вместе с ним всегда показывайте precision, recall и выбранный порог. Без заданной стоимости ошибок порог `0.5` нужен только для сопоставимой учебной демонстрации.